Hands-On Machine Learning

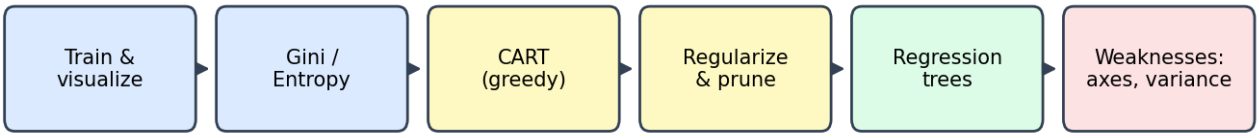
Chapter 6 — Decision Trees

CART · Gini & Entropy · Regularization · Regression Trees

ملخص كشكولك + تكملة النقط الناقصة + رسومات وأكواد للمراجعة

من كشكولك | إضافة لإكمال النقص | انتبه / تصحيح

كل الأرقام في مربعات **Output (real run)** الخضرا طلعت من تشغيل الكود فعلًا (Iris الحقيقية وmoons وبيانات الـ regression بنفس إعدادات الكتاب). الرسمة بتاعة الـ Graphviz اتولّدت بنفس كودك (`Source` + `export_graphviz`).



خريطة الشابتر.

1 يعني إيه Decision Tree؟

من كشكولك

يعني إيه Decision Tree؟ موديل بيتعلم من **مجموعة أسئلة متتابعة** عن الـ Features وكل إجابة بتودّي لفرع مختلف لحد ما يوصل لقرار نهائي.

→ تخيل إن عندك بيانات زهور Iris وعايز الموديل يحدد نوع الزهرة من قياسات البتلات. الشجرة ممكن تبدأ بالسؤال: **هل طول البتلة أقل من أو يساوي قيمة معينة؟** (لو أه روح ناحية، ولو لأ تنتقل لسؤال تاني).

→ الشجرة تقدر تعمل Classification لتحديد فئة أو Regression لتوقع قيمة رقمية. وممكن كمان تتعامل مع أكتر من Target في بعض المهام وده اسمه Multioutput. وهي كمان من المكونات الأساسية لـ Random Forest اللي هيتم دراسته في الشابتر القادم.

→ **الفكرة:** بدل ما يكون عندنا معادلة واحدة معقدة، عندنا **سلسلة شروط واضحة**: سؤال ثم فرع، ثم سؤال آخر، ثم نتيجة.

إضافة

(الـ Multioutput جربناه فعلًا: درّبنا شجرة واحدة على target من عمودين: نوع الزهرة + هل طول البتلة أكبر من 4.5.)

```
y2 = np.c_[iris.target, iris.data["petal length (cm)"] > 4.5]     # 2 targets per flower
mo = DecisionTreeClassifier(max_depth=3, random_state=42).fit(iris.data, y2)
print(mo.predict(iris.data.iloc[[0, 100]]))
```

Output (real run)

```
[[0 0]
 [2 1]]      # shape (2, 2) for 2 flowers: [species, long_petal?] each
```

2 تدريب الشجرة ورسمها (Training and Visualizing)

من كشكولك

هنستخدم Dataset اسمها Iris وفيها **150** زهرة من 3 أنواع. هنختار ميزتين فقط للتدريب: **طول البتلة وعرض البتلة**.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target

tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
print(tree_clf.score(X_iris, y_iris), tree_clf.tree_.node_count, tree_clf.get_n_leaves())
```

Output (real run)

```
0.9600 5 3      # accuracy, nodes (1 root + 1 inner + 3 leaves), leaves
```

من كشكولك

- `X_iris` : فيها قيم الـ Features اللي الموديل هيستعملها لاتخاذ القرارات. كل صف يمثل زهرة والعمودان هما طول البتلة وعرضها.
- `y_iris` : فيها النوع الصحيح لكل زهرة، أي الـ Target اللي عايزين الشجرة تتعلم تتنبأ بيه.
- `DecisionTreeClassifier` : هو موديل الشجرة المستخدم في التصنيف (Classification).
- `max_depth` : معناها **أقصى عمق** للشجرة، عشان نخلي الشجرة بسيطة ويسهل فهمها.
- `fit(X_iris, y_iris)` : هي **خطوة التدريب**: الشجرة بتبحث عن الأسئلة المناسبة للفصل بين أنواع الزهور.

من كشكولك

رسم الشجرة باستخدام Graphviz

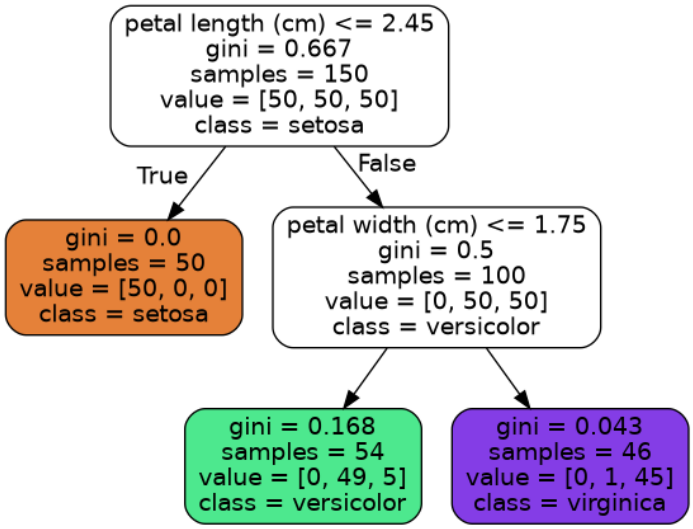
بعد التدريب نقدر نعرض الأسئلة والفروع باستخدام `export_graphviz` ثم نفتح الرسم الناتج:

```
from sklearn.tree import export_graphviz
from graphviz import Source

export_graphviz(tree_clf, out_file="iris_tree.dot",
                feature_names=["petal length (cm)", "petal width (cm)"],
                class_names=iris.target_names, rounded=True, filled=True)
Source.from_file("iris_tree.dot")
```

Output (real run)

```
digraph Tree {
node [shape=box, style="filled, rounded", color="black", fontname="helvetica"] ;
edge [fontname="helvetica"] ;
0 [label="petal length (cm) <= 2.45\ngini = 0.667\nsamples = 150\nvalue = [50, 50, 50]\nclass = setosa",
fillcolor="#ffffff"] ;
1 [label="gini = 0.0\nsamples = 50\nvalue = [50, 0, 0]\nclass = setosa", fillcolor="#e58139"] ;
0 -> 1 [labeldistance=2.5, labelangle=45, headlabel="True"] ;
...
```



الشجرة الحقيقية اللي طلعت من كودك بالظبط (Graphviz): اللون = الفئة الأغلب في العقدة، وكل ما كان أغمق كان أنقى.

من كشكولك

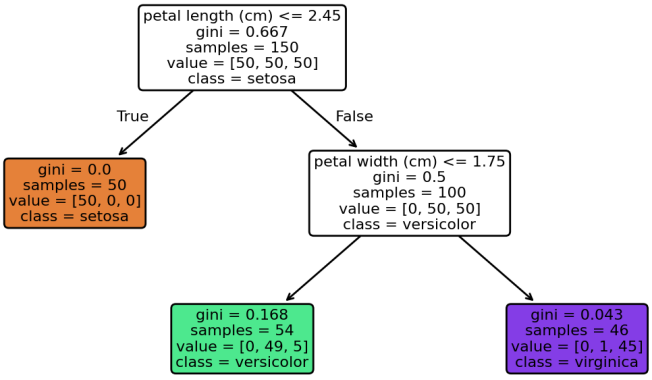
→ `export_graphviz()` بتكتب ملف الشجرة في ملف بصيغة DOT، فـ `Source.from_file()` بتعرض الملف كرسم في بيئة تدعم Graphviz.

- **Root** هي أول عقدة في الشجرة.
- **Node** هي عقدة في الشجرة فيها اختبار أو قرار.
- **Leaf** هي نهاية الفرع ومنها يصير التنبؤ.

إضافة

العقدة	سؤالها	samples	value	gini	النوع
0 (root)	petal length ≤ 2.45	150	[50 ,50 ,50]	0.667	inner node
1	—	50	[0 ,0 ,50]	0.000	leaf: setosa (pure!)
2	petal width ≤ 1.75	100	[50 ,50 ,0]	0.500	inner node
3	—	54	[5 ,49 ,0]	0.168	leaf: versicolor
4	—	46	[45 ,1 ,0]	0.043	leaf: virginica

الجدول ده طالع من `tree_clf.tree_` مباشرة (العقد الـ 5 اللي في الرسمة).



نفس الشجرة بـ `plot_tree` (matplotlib) لو مفيش Graphviz عندك: `plot_tree(tree_clf, feature_names=..., class_names=..., filled=True)`.

من كشكولك

- **Samples:** عدد عينات التدريب التي وصلت إلى العقدة بعد تطبيق الشروط السابقة. مثلًا لو `samples=54` فده معناه إن 54 زهرة من بيانات التدريب وصلت إلى هذه العقدة.
- **Value:** عدد العينات من كل class الموجود في العقدة. لو كانت `Value = [0, 49, 5]` فده معناه: لا توجد عينات من النوع الأول، و49 من النوع الثاني، و5 من النوع الثالث، المجموع 54.
- **Gini:** مقياس لمدى **اختلاط الفئات** داخل العقدة. لو كل العينات من فئة واحدة تكون قيمة Gini مساوية للصفر، أي إن العقدة نقية (pure).

معادلة Gini Impurity:

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

حيث `p(i,k)` هي نسبة العينات من الفئة `k` داخل العقدة `i`. → كلما اقتربت Gini من الصفر كانت العقدة أنقى. أما ارتفاعها فيعني إن الفئات أكثر اختلاطًا. القيمة الدقيقة الممكنة تعتمد على عدد الفئات وتوزيعها.

```
def gini(counts):
    p = np.array(counts) / sum(counts)
    return 1 - (p ** 2).sum()

print(gini([0, 49, 5]))      # the 54-sample leaf
print(gini([0, 50, 0]))      # a pure node
print(gini([50, 50, 50]))    # the root: 3 classes mixed equally
```

```
Output (real run)
0.1680
0.0000
0.6667      # = 1 - 3 x (1/3)² = 2/3, the MAXIMUM for 3 classes
```

انتبه / تصحيح

في كشكولك كتبت إن Gini للعقدة `[0, 49, 5]` "تساوي تقريبًا 0.268". الحساب الصحيح: `1 − (49/54)² − (5/54)²` = **0.168** (وده نفس الرقم اللي في رسمة Graphviz فوق: `gini = 0.168`). يبدو إنك قلبت 1 مع 2 وأنت بتكتب.

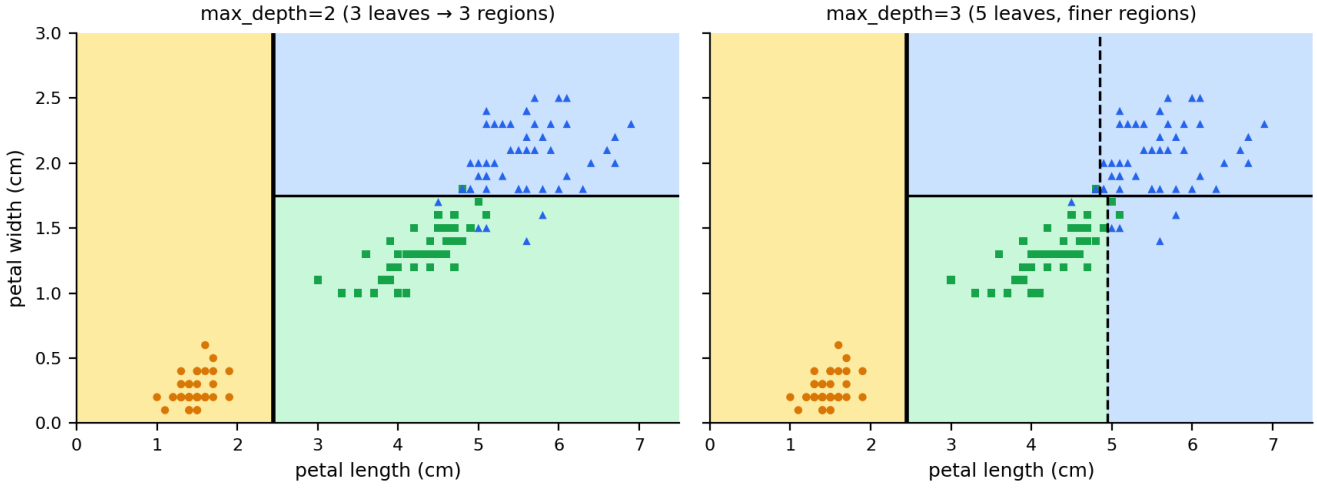
وبالمناسبة: القيمة القصوى لـ Gini مع 3 فئات هي 2/3 (0.667) مش 1، ومع فئتين 0.5.

3 حدود القرار (Decision Boundaries)

من كشكولك

الشجرة **لا تتعلّم الأسئلة فقط**، لكنها أيضًا **بتقسّم مساحة البيانات إلى مناطق**، وبما إننا نستخدم ميزتين، نستطيع رسم هذه المناطق على مستوى ثنائي الأبعاد.

→ عند استخدام `max_depth=2` تكون الشجرة محدودة العمق، فلا تستطيع إجراء عدد كبير من التقسيمات. لو سمحنا لها بعمق أكبر مثل `max_depth=3` يمكنها إضافة شروط أخرى وتقسيم المناطق إلى أجزاء أصغر وبالتالي **تتغير حدود القرار**.



حدود القرار الحقيقية لنفس الداتا: depth=2 (شمال): 3 مناطق، الخط الأسود السميك هو سؤال الـ root والرفيع سؤال العمق الثاني. depth=3 (يمين): 5 مناطق (الخط المتقطع = سؤال جديد بعمق 3) ودقة التدريب 0.973 بدل 0.960.

من كشكولك

→ **ميزة الأشجار هنا:** إنها **لا تحتاج إلى تطبيع أو توحيد قيم الـ Features** لأن كل اختبار مثل `petal length ≤ 2.45` **لا يعتمد على المسافة** بين القيم أو على كونها في نفس المقياس.

```
scaler = StandardScaler().fit(X_iris)
tree_scaled = DecisionTreeClassifier(max_depth=2, random_state=42).fit(scaler.transform(X_iris), y_iris)

print((tree_scaled.predict(scaler.transform(X_iris)) == tree_clf.predict(X_iris)).all())
print(tree_clf.tree_.threshold[0], "->", tree_scaled.tree_.threshold[0] * scaler.scale_[0] +
scaler.mean_[0])
```

```
Output (real run)
True
2.45 -> 2.45      # same split, just expressed in scaled units
```

إضافة

تجربة حقيقية تأكد كلامك: قارنت الشجرة على الداتا الأصلية والشجرة على الداتا بعد `StandardScaler` : **نفس التنبؤات بالظبط** ونفس عتبة الـ split (2.45 بعد ما رجّعتها لوحداتها). عكس الـ SVM اللي شفناه في شابتر 5 (اللي scaling عنده ضروري).

4 الـ tree_ و White Box مقابل Black Box

من كشكولك

بعد تدريب شجرة يتيح لك scikit-learn الوصول إلى معلومات بنيتها عن طريق الخاصية `tree_` (`tree_clf.tree_`)، وهي تحتوي على معلومات داخلية عن **العقد والتقسيمات**، ويمكن الاستفادة منها عند فحص طريقة عمل الموديل.

→ الشجرة في العادة تُعد **White Box Model** لأننا نستطيع عرض الأسئلة والفروع التي أدّت إلى القرار وقراءتها. على النقيض، توجد موديلات أقرب إلى **Black Box** حيث يكون تفسير القرار من داخل الموديل أصعب بكثير.

→ لكن سهولة قراءة الشجرة **لا تضمن وحدها** إن قراراتها صحيحة أو عادلة، وإنما تعني إن خطوات القرار **ظاهرة وقابلة للفحص**.

```
from sklearn.tree import export_text
print(export_text(tree_clf, feature_names=["petal length (cm)", "petal width (cm)"]))

t = tree_clf.tree_
print(t.node_count, t.max_depth, t.children_left, t.children_right, t.feature)
```

Output (real run)

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2
5 2 [1, -1, 3, -1, -1] [2, -1, 4, -1, -1] [0, -2, 1, -2, -2]
```

إضافة

`export_text` بيطبعلك الشجرة كـ **قواعد if/else** وده بالظبط معنى "White Box": تقدر تقرا القرار بالنص وتشرحه لشخص مش تقني. والـ `-1` في `children_left` معناها "ده leaf مفيش أبناء".

وللتفسير برضه: `feature_importances_` (مجموع نقص الـ impurity اللي كل feature عملته). على Iris بأربع features وعمق 3:

Output (real run)

```
sepal length (cm)   0.0
sepal width (cm)    0.0
petal length (cm)   0.586
petal width (cm)    0.414
```

الشجرة اعتمدت على مقاييس البتلة بس: أهمية الـ sepal = **0 بالظبط** (ما استُخدمتش في أي سؤال)، و petal length = 0.586 و petal width = 0.414. ده بالظبط اللي كنا شايفينه من الرسمة.

5 تقدير الاحتمالات (Estimating Class Probabilities)

من كشكولك

الشجرة لا تستطيع فقط إن تتنبأ بنوع الزهرة، يمكنها أيضًا **تقدير احتمال انتمائها إلى كل فئة** باستخدام `predict_proba()`:

```
print(tree_clf.predict_proba([[5, 1.5]]))
print(tree_clf.predict([[5, 1.5]]))
```

Output (real run)

```
[[0.000 0.907 0.093]]
[1]
```

من كشكولك

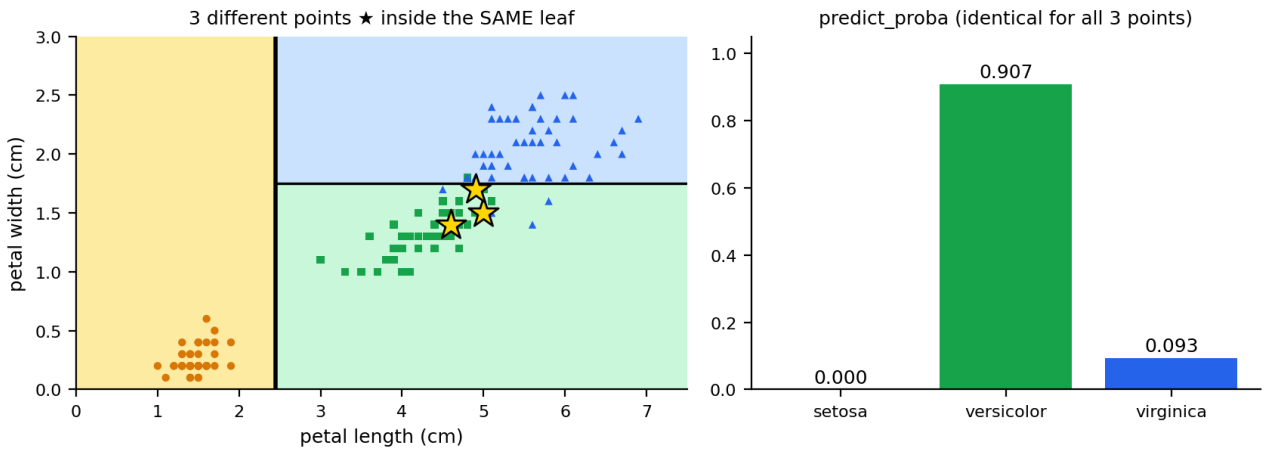
المدخل `[5, 1.5]` يمثل زهرة طول بتلتها 5 سم وعرضها 1.5. تتبع الشجرة الشروط حتى تصل إلى **ورقة فيها 54 عينة تدريبية**: 49 من النوع الثاني و5 من النوع الثالث ولا يوجد من النوع الأول. إذن الاحتمالات تُحسب من **النسب داخل الورقة**:

$$\hat{p}_k = \frac{\text{samples of class } k \text{ in the leaf}}{\text{samples in the leaf}}, \qquad \hat{y} = \text{argmax}_k \, \hat{p}_k$$

- `P(Setosa) = 0/54 = 0`
- `P(Versicolor) = 49/54 ≈ 0.907`
- `P(Virginica) = 5/54 ≈ 0.093`

بالتالي التنبؤ يكون هو **Class 1** أي Versicolor لأنه صاحب أعلى احتمال.

نقطة مهمة: أي نقطة تقع في **الورقة نفسها** تحصل على **الاحتمالات نفسها** في هذه الشجرة، حتى لو اختلفت قيم الـ Features الخاصة بها ما دامت وصلت إلى الورقة نفسها.



تجربة حقيقية: 3 نقاط مختلفة (النجوم) جوه نفس الورقة بتاخد احتمالات متطابقة [0.0, 0.907, 0.093]. شوف `[4.6, 1.4]` = [0.0, 0.907, 0.093] و `[4.9, 1.7]` = [0.0, 0.907, 0.093].

إضافة

ده معناه إن احتمالات الشجرة **خشنة**: مفيش "تدرّج" جوه الورقة (نقطة قريبة من الحد زي نقطة في نص الورقة). وهي برضه احتمالات مبنية على عدد قليل من العينات (54)، فمتعتمدش عليها زي ما بتعتمد على احتمالات موديل ناعم زي Logistic Regression.

6 خوارزمية CART (كيف تتعلم الشجرة؟)

من كشكولك

Scikit-learn تستخدم خوارزمية اسمها **CART** (Classification and Regression Tree) لبناء الشجرة، والفكرة الأساسية إنها تختار في كل عقدة **Feature وحدًّا فاصلًا (Threshold)** لتقسيم البيانات إلى مجموعتين.

→ مثلًا قد تقارن بين أسئلة كثيرة محتملة: هل طول البتلة أقل من 2.45 سم؟ هل أقل من 5؟ ... الخوارزمية تختار السؤال الذي **يقلل عدم نقاء المجموعتين الناتجتين بأفضل شكل** وفق دالة التكلفة.

→ **دالة تكلفة التقسيم في Classification:**

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

- `k` : الـ Feature التي تختارها، `t_k` : هو الـ Threshold الذي سنقسم عنده.
- `m` : عدد العينات في العقدة الحالية، و`m_left` , `m_right` هما عدد العينات في الفرع الأيسر والفرع الأيمن، و`G_left` , `G_right` هما قيمتا عدم النقاء في الفرعين.

نحسب **متوسط عدم النقاء موزونًا بعدد العينات**، لذلك لو كان فرع فيه 100 عينة والآخر 5 عينات فالفرع الأكبر سيكون له وزن أكبر في الحساب.

إضافة

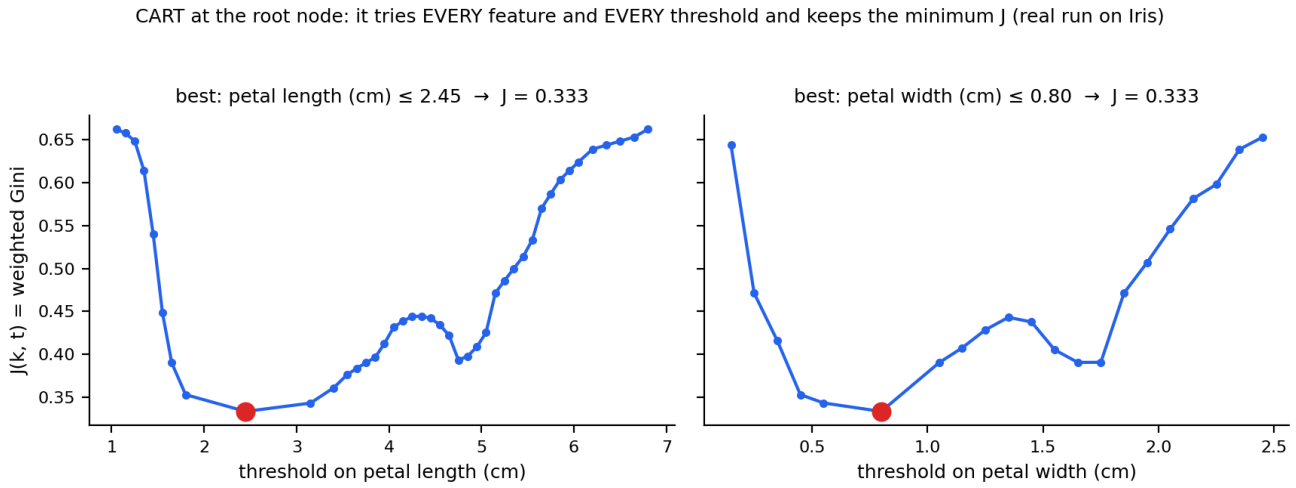
مثال بالأرقام على كلامك (100 عينة و5 عينات): لو الفرع الكبير gini = 0.5 والصغير 0 (نقي): `J = 100/105 × 0.5 + 5/105 × 0 =` **0.476**. يعني الفرع الصغير بيأثر بـ 5% بس من القرار. وعلى الـ root بتاع Iris: الفرع الأيسر (50 setosa) gini = 0، والأيمن (100) gini = 0.5 → `J = 50/150 × 0 + 100/150 × 0.5 =` **0.333** (وده أحسن J لقيته الخوارزمية).

```
def split_cost(X, y, k, t):
    left, right = X[:, k] <= t, X[:, k] > t
    g = lambda yy: 1 - sum((yy == c).mean() ** 2 for c in np.unique(y))
    return left.mean() * g(y[left]) + right.mean() * g(y[right])

for k, name in enumerate(["petal length", "petal width"]):
    values = np.unique(X_iris[:, k]); thresholds = (values[:-1] + values[1:]) / 2   # try EVERY midpoint
    J = [split_cost(X_iris, y_iris, k, t) for t in thresholds]
    print(name, "best threshold =", round(thresholds[np.argmin(J)], 2), " J =", round(min(J), 3))
```

Output (real run)

```
petal length best threshold = 2.45  J = 0.333
petal width best threshold = 0.80  J = 0.333
```



ده بالظبط اللي CART بتعمله في الـ root: بتجرّب كل feature وكل threshold وبتاخد أقل J. (تشغيل حقيقي على Iris)

إضافة

اكتشاف مهم من التجربة: الـ feature الاتنين طلّعوا نفس أقل J (0.333)! يعني فيه **تعادل** بين `petal length ≤ 2.45` و `petal width ≤ 0.8` (الاتنين بيفصلوا setosa تمامًا). والـ sklearn بيكسر التعادل **عشوائيًا** (حسب `random_state`)، وده سبب ظهور شجر مختلف في القسم 12.

من كشكولك

ليه اسمها Binary Tree؟ في CART كل عملية تقسيم تنتج **فرعين فقط** (أولًا). يعني حتى لو كان عندنا ثلاث فئات أو عشر فئات، كل سؤال في الشجرة يعطي فرعين. بعض خوارزميات الأشجار الأخرى مثل ID3 في صورتها التقليدية يمكنها إنتاج **أكثر من فرع** من العقدة الواحدة حسب قيم الـ Feature المستخدمة في التقسيم.

من كشكولك

Greedy Algorithm (الاختيار الطماع): CART تتعلم بطريقة Greedy: تختار **أفضل تقسيم للعقدة الحالية** وفق المعيار كل مرة، ثم تكرر العملية على العقد الناتجة، لكنها **لا تجرب جميع أشكال الأشجار الممكنة** لكي تضمن الحصول على أفضل شجرة نهائية على مستوى البيانات كلها.

إيجاد الشجرة المثلى عالميًا مسألة شديدة الصعوبة حسابيًا؛ ويوضح الكتاب إن العثور على أفضل شجرة قرار يُعد **NP-complete** (وقت الحل يزيد مع حجم المشكلة `O(exp(m))`) ولذلك تُستخدم الخوارزمية الطماعة (Greedy) حلًّا مقبولًا **بدل البحث الشامل**.

وتستمر CART في تقسيم الفروع إلى أن يتحقق شرط توقف مثل الوصول إلى `max_depth` أو عدم وجود تقسيم مناسب يخفض دالة التكلفة وفق الإعدادات المستخدمة.

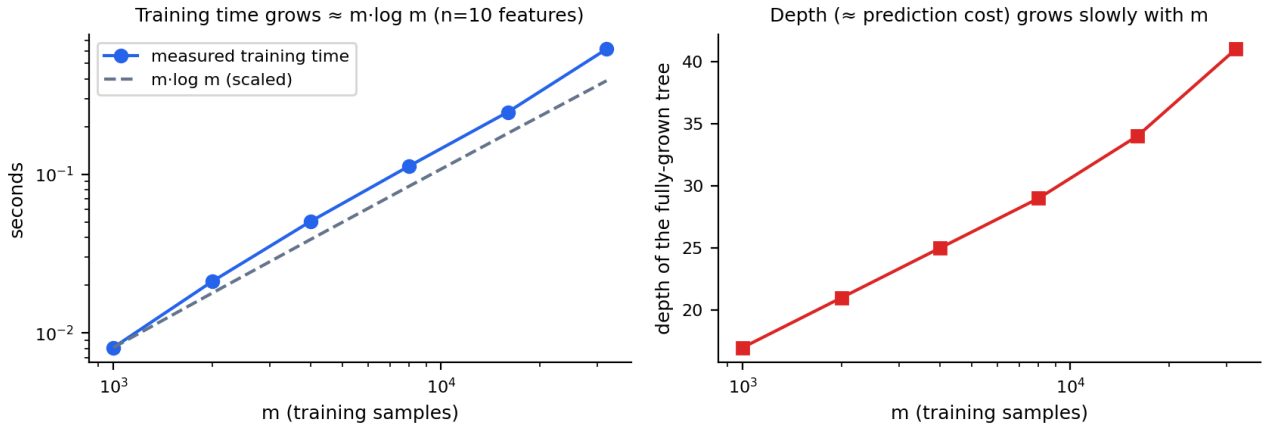
7 التعقيد الحسابي (Computational Complexity)

من كشكولك

لو عندنا `m` عينة تدريب والشجرة **متوازنة تقريبًا**، فإن تحقق عملية التنبؤ تتطلب عادةً المرور عبر عدد من العقد يتناسب مع **عمق الشجرة** أي تقريبًا `O(log₂(m))` في حالة الشجرة المتوازنة.

→ لو الشجرة **غير متوازنة بشدة** قد تصبح عملية التنبؤ أبطأ، لأن التعقيد يعتمد على **عدد المستويات التي يمر بها المسار الفعلي** وليس على عدد العينات وحده.

→ أما **تدريب** الشجرة باستخدام البحث عن أفضل Threshold لكل Feature في كل عقدة فالتعقيد التقريبي الذي يورده الكتاب هو `O(n × m log m)` (`n` = عدد الـ Features، `m` = عدد العينات). الصياغة ليست دائمًا خطية تمامًا بسبب عوامل `log m`.



تجربة حقيقية (n=10 features، داتا فيها noise): شمال: وقت التدريب فعلًا ماشي مع `m·log m` تقريبًا (الخط المتقطع). يمين: عمق الشجرة (= تكلفة التنبؤ).

انتبه / تصحيح

من القياس الحقيقي: لما m زادت من 1,000 لـ 32,000 عمق الشجرة بقى **41** مع إن `log₂(32,000) ≈ 15` . السبب إن الشجرة اللي بتكبر من غير قيود على داتا فيها noise **مش متوازنة** (زي ما قلت في كلامك: التعقيد بيعتمد على العمق الفعلي). ورغم كده التنبؤ سريع جدًا: 59 ميكروثانية للعينة الواحدة بعد التدريب على 32,000 عينة.

8 الـ Entropy و الـ Gini Impurity

من كشكولك

حتى الآن استخدمنا Gini لكن توجد **طريقة أخرى شائعة اسمها Entropy** وهي تقيس **عدم اليقين** داخل العقدة.

$$H_i = -\sum_{k=1,\ p_{i,k}>0}^{n} p_{i,k} \log_2(p_{i,k})$$

→ في المعادلة `p(i,k)` هي نسبة الفئة `k` في العقدة `i` . إذا كانت العقدة تحتوي على فئة واحدة فقط تصبح Entropy مساوية للصفر لأنه لا عدم يقين فيها.

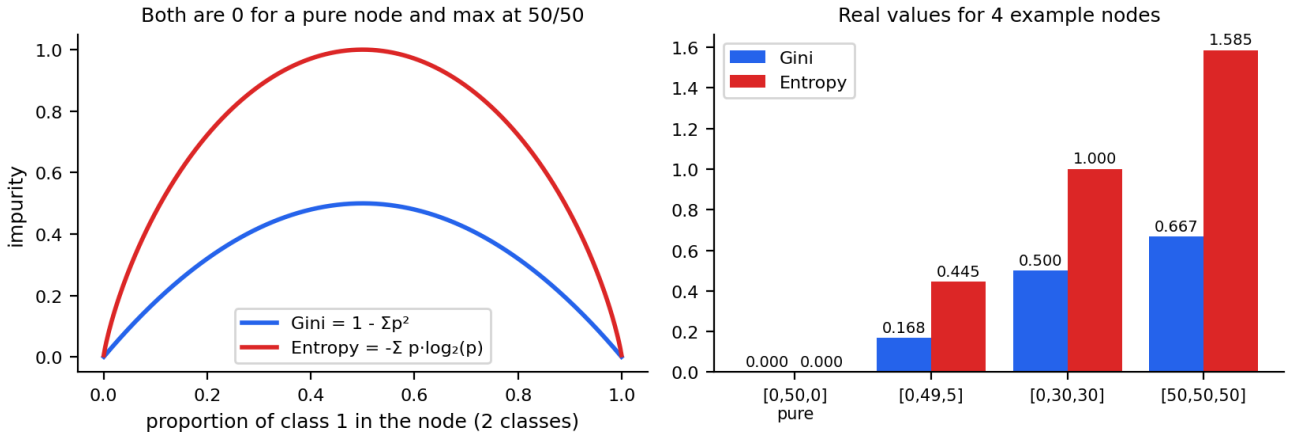
→ على سبيل المثال: عند وجود `[0, 49, 5]` في عقدة مجموع عيناتها 54 تكون Entropy ≈ **0.445** بينما Gini لهذه العقدة تساوي تقريبًا **0.168**.

```python
def entropy(counts):
    p = np.array(counts) / sum(counts); p = p[p > 0]
    return -(p * np.log2(p)).sum()
print(entropy([0, 49, 5]), entropy([50, 50, 50]))

tree_ent = DecisionTreeClassifier(criterion="entropy", max_depth=2, random_state=42)
```

Output (real run)

```
0.4451 1.5850      # entropy of [0,49,5] and of the root (log2(3) = 1.585)
```



شمال: الاتنين بيبدأوا من 0 (نقية) وأقصى قيمة عند 50/50؛ الـ entropy أعلى ومنحناها أحدّ. يمين: قيم حقيقية لـ 4 عقد.

من كشكولك

→ في scikit-learn يستخدم `DecisionTreeClassifier` معيار **Gini افتراضيًا**، ولو أردنا استخدام Entropy نستطيع كتابة `criterion="entropy"`.

→ غالبًا ما يؤدي المعياران إلى أشجار متشابهة، لكن قد يختلف شكل الشجرة الناتجة. حساب Gini أبسط عادةً وأسرع قليلًا، كذلك يميل Gini إلى عزل الفئة الأكثر تكرارًا داخل فرع في بعض الحالات، في حين قد ينتج Entropy تقسيمًا أكثر توازنًا. هذا اتجاه عام وليس قاعدة مطلقة في كل Dataset.

إضافة

تجربة حقيقية على كلامك: (1) على Iris بعمق 3 المعياران طلّعوا **نفس الشجرة بالظبط** (نفس القواعد) ودقة 0.973. (2) لكن على 40 داتا moons مختلفة طلعت الأشجار **مختلفة في 36 من 40** حالة (بنفس العمق). يعني "متشابهين" في الأداء مش في الشكل. (3) السرعة: مقاس تدريب شجرة على 20,000 عينة × 20 feature: gini = 0.58 ث و entropy = 0.46 ث؛ يعني **ما لقيتش فرق ثابت** في السرعة في تجربتي (الفرق ضمن الـ noise)، فالأفضل تختار بالـ validation مش بالسرعة.

9 الـ Regularization (تقليل الـ Overfitting)

من كشكولك

شجرة القرار تُعد من الموديلات **غير المعلمية (Nonparametric)** بالمعنى المعتاد (ليس لدينا عدد ثابت من معاملات النموذج نحدده قبل التدريب، وإنما شكل الشجرة وحجمها يتحدد بالبيانات نفسها).

→ **الميزة** إنها تستطيع تمثيل أنماط معقدة، لكن **المشكلة** إن الشجرة لو تُركت من دون قيود قد تتعلم تفاصيل صغيرة وضوضاء خاصة ببيانات التدريب، عندها يمكن أن تحقق أداءً ممتازًا على بيانات التدريب لكن أداءها يكون أضعف على بيانات جديدة. وهذه هي مشكلة الـ overfitting.

→ **أهم Hyperparameters:**

1. `max_depth` : يحدد أقصى عمق للشجرة. تقليل القيمة يحد من تعقيدها.
2. `min_samples_split` : يحدد أقل عدد عينات يجب أن تحتويه العقدة حتى يُسمح بمحاولة تقسيمها.
3. `min_samples_leaf` : يحدد أقل عدد عينات يجب أن يبقى في كل leaf بعد التقسيم. رفع هذا الرقم يمنع إنشاء أوراق صغيرة جدًا.
4. `min_weight_fraction_leaf` : يحدد الحد الأدنى لوزن العينات في الورقة كنسبة من مجموع أوزان العينات.
5. `max_leaf_nodes` : يحدد أقصى عدد من الأوراق في الشجرة.
6. `max_features` : يحدد عدد الـ Features التي تؤخذ في الاعتبار عند البحث عن أفضل تقسيم في العقدة، وفق القيمة المحددة.

القاعدة العامة: زيادة القيود مثل تقليل `max_depth` أو `max_leaf_nodes` أو رفع `min_samples_split` / `min_samples_leaf` تجعل الشجرة أبسط وغالبًا تساعد على تقليل الـ overfitting، لكن زيادة القيود أكثر من اللازم قد تسبب Underfitting.

من كشكولك

مثال الكتاب: Moons:

```
from sklearn.datasets import make_moons
from sklearn.metrics import accuracy_score

X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
tree_unregularized = DecisionTreeClassifier(random_state=42)
tree_regularized = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_unregularized.fit(X_moons, y_moons)
tree_regularized.fit(X_moons, y_moons)

X_test, y_test = make_moons(n_samples=1000, noise=0.2, random_state=43)
pred_unregularized = tree_unregularized.predict(X_test)
pred_regularized = tree_regularized.predict(X_test)
print(accuracy_score(y_test, pred_unregularized))
print(accuracy_score(y_test, pred_regularized))
```

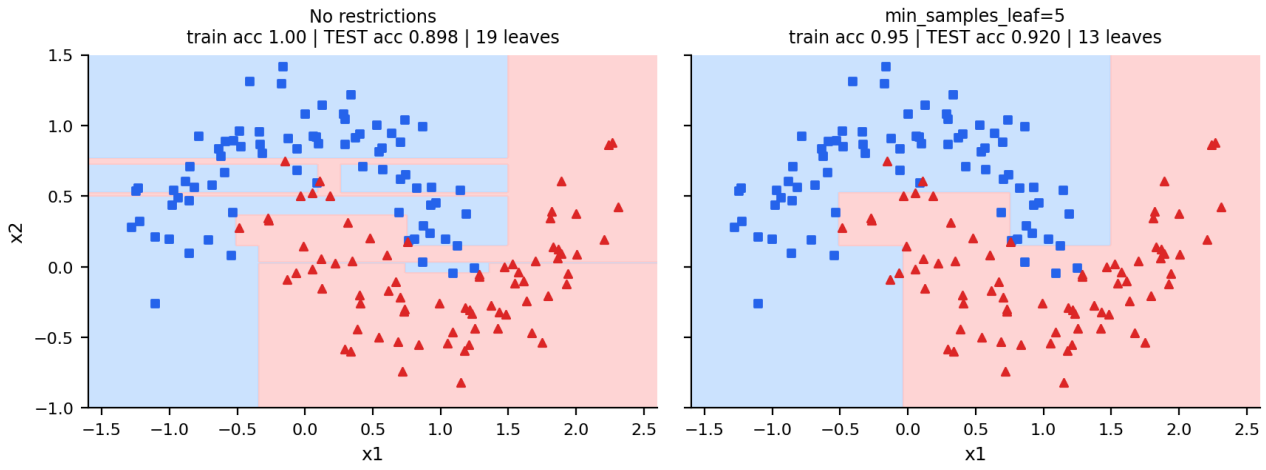
Output (real run)

```
0.898
0.920
```

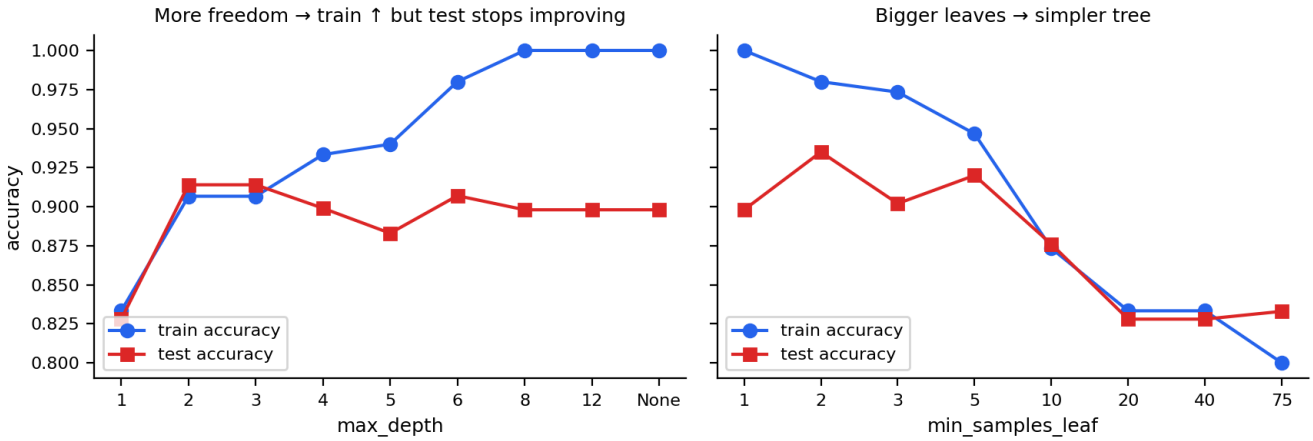
من كشكولك

→ في المثال: حققت الشجرة غير المقيدة Accuracy مقدارها **0.898** بينما حققت الشجرة التي تستخدم `min_samples_leaf=5` مقدار **0.92** على مجموعة الاختبار المستخدمة في الكتاب. (والتشغيل الحقيقي طلّع نفس الرقمين بالظبط.)

→ **المغزى:** ليس إن إضافة القيود سترفع الدقة دائمًا، بل إن الشجرة الأولى يمكنها إنشاء فروع وأوراق كثيرة لتناسب تفاصيل بيانات التدريب، أما القيد فيمنع بعض هذه التقسيمات ويجعل النموذج أكثر قدرة على التعميم هنا.



الشجرة غير المقيدة (19 leaf، دقة تدريب 1.00) بتعمل مستطيلات صغيرة حوالين نقاط معزولة. المقيدة (13 leaf) أنعم ودقتها على الاختبار أعلى.



تجربة حقيقية على نفس الداتا: شمال: كل ما سمحنا بعمق أكبر دقة التدريب طلعت لـ 1.00 لكن دقة الاختبار وقفت (ورجعت نزلت) بعد عمق 2-3. يمين: `min_samples_leaf` أكبر = شجرة أبسط، لكن لو كبّرناه أكتر من اللازم (+20) الاتنين بينزلوا (underfitting).

إضافة

لاحظ في اليمين: أحسن دقة اختبار طلعت عند `min_samples_leaf=2` (0.935) مش عند 5 (0.920)، ومع 75 بتنزل لـ 0.833 (الشجرة بقت 2 leaves بس). يعني **مفيش قيمة سحرية**: بنختارها بالـ validation (`GridSearchCV`) زي ما اتعلمنا.

10 تقليم الشجرة (Pruning)

من كشكولك

→ **Pruning (تقليم الشجرة):** طريقة أخرى لتقليل التعقيد هي إن نسمح للشجرة **بالنمو أولًا** ثم **نحذف الفروع** التي لا تقدّم تحسنًا ذا دلالة إحصائية في نقاء العقد. هذا هو مبدأ الـ Pruning.

→ يذكر الكتاب استخدام اختبار مثل χ^2 لتقدير ما إذا كان التحسن الظاهر في التقسيم قد يكون **مجرد صدفة**. يُستخدم الـ p-value للحكم على ذلك، وإذا كانت قيمته أكبر من حد معين (مثل **5%** في المثال الذي يذكره الكتاب)، قد يُعتبر التحسن غير كافٍ وتُحذف العقدة وأبناؤها، وتتكرر العملية حتى لا تبقى عقد غير ضرورية بحسب الاختبار.

→ الـ Pruning إذن أسلوب آخر لضبط تعقيد الشجرة **بجانب التحكم في Hyperparameters أثناء التدريب**.

إضافة

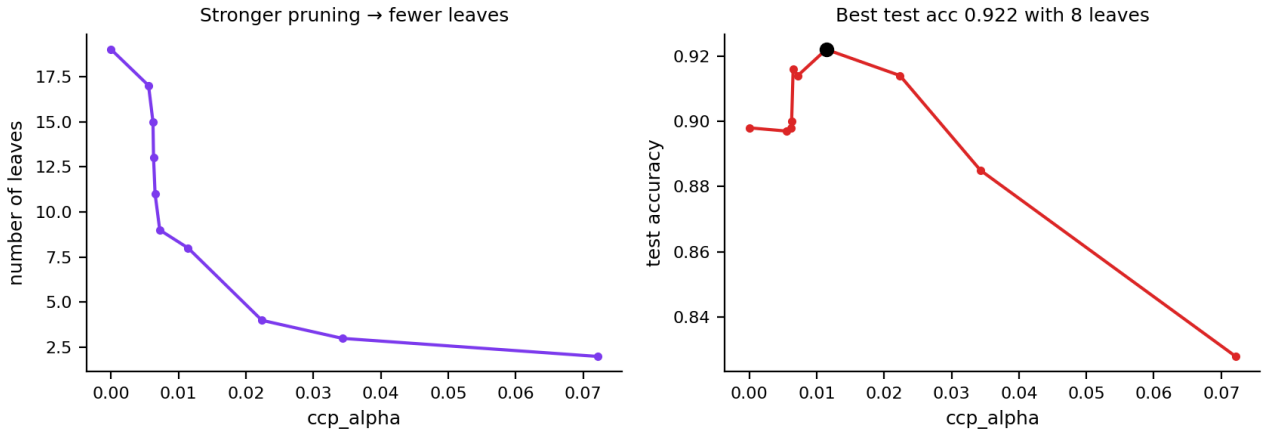
انتبه: الـ scikit-learn **مش بتنفّذ اختبار χ^2 بتاع الكتاب**. اللي موجود فيها شكل تاني من الـ pruning اسمه Cost-Complexity Pruning: بنحط عقوبة α على عدد الأوراق، ونحذف الفروع اللي تحسينها أقل من العقوبة:

$$R_\alpha(T) = R(T) + \alpha\,|\tilde{T}| \qquad (|\tilde{T}| = \text{number of leaves})$$

```
path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_moons, y_moons)
for alpha in path.ccp_alphas[:-1]:
    t = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha).fit(X_moons, y_moons)
    print(round(alpha, 4), t.get_n_leaves(), round(accuracy_score(y_test, t.predict(X_test)), 3))
```

```
Output (real run)
0.0000  19  0.898
0.0062  15  0.898
0.0065  11  0.916
0.0114   8  0.922
0.0343   3  0.885
...
```

(بنقرا: `ccp_alpha` ، عدد الأوراق، دقة الاختبار.) أحسن نتيجة: **0.922** عند 8 أوراق بس (بدل 19 في الشجرة الكاملة بدقة 0.898). ودي نفس فكرتك: تقليم الأفرع الضعيفة بعد النمو.



تجربة حقيقية: كل ما زودنا `ccp_alpha` الأوراق بتقل، ودقة الاختبار بتتحسن لحد نقطة (النقطة السودا) وبعدها بتنزل (التقليم الزيادة = underfitting).

11 Decision Tree Regression

من كشكولك

حتى هذه النقطة استخدمنا الشجرة **للتصنيف فقط**، لكن يمكن استخدامها أيضًا في الـ Regression أي توقع قيمة رقمية مستمرة عن طريق `DecisionTreeRegressor`.

```
from sklearn.tree import DecisionTreeRegressor

X_quad = np.random.rand(200, 1) - 0.5
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
print(tree_reg.predict([[0.2]]))
```

Output (real run)

```
[0.0277]      # R² on the training data = 0.797
```

من كشكولك

→ هنا `X_quad` و`y_quad` هما بيانات الـ Regression التي أنشأها مثال الكتاب. المتغير `max_depth=2` يقيّد عمق الشجرة كما فعلنا في التصنيف، لكن **الاختلاف الأساسي** هو طريقة تقييم جودة التقسيم والنتيجة التي تنتجها الورقة.

→ في الـ Regression، **تُنتج كل ورقة متوسط قيم الـ Target** لعينات التدريب التي وصلت إليها، أي إن الشجرة **لا تختار اسم class** كما يحدث في Classification؛ بل تنتج **قيمة رقمية**.

→ **دالة التكلفة في Regression:**

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

بدلًا من تقليل Gini أو Entropy، يحاول التقسيم تقليل **متوسط مربع الخطأ (MSE) الموزون** في الفرعين الناتجين.

$$\text{MSE}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} (y^{(i)} - \hat{y}_{\text{node}})^2 , \qquad \hat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)}$$

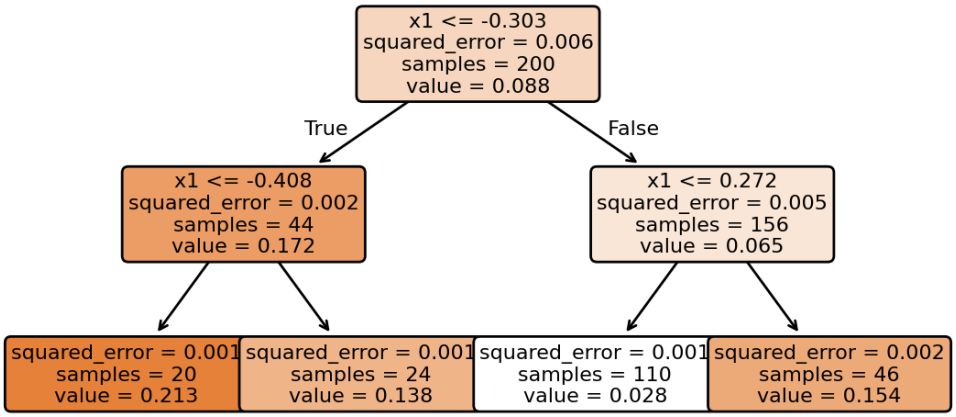
حيث `ŷ_node` هو **متوسط الـ Target داخل العقدة** و`y(i)` هي القيمة الفعلية.

إضافة

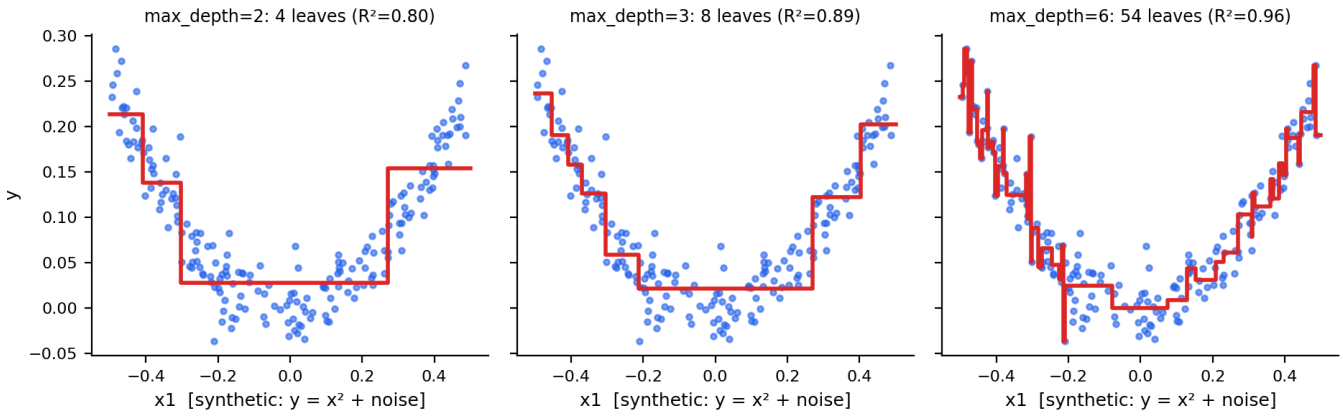
بالأرقام الحقيقية: الـ root قسّم على `x1 ≤ -0.303` (200 عينة، MSE = 0.0061، متوسط الـ target = 0.088). الفرع الأيسر 44 عينة (MSE 0.0024) والأيمن 156 عينة (MSE 0.0046). التكلفة `J = 44/200 × 0.0024 + 156/200 × 0.0046 =` **0.00413**. وده نفس الـ "squared_error" اللي بتشوفه في الرسمة تحت (اسمه `squared_error` بدل gini).

Output (real run)

```
|--- x1 <= -0.30
|   |--- x1 <= -0.41
|   |   |--- value: [0.21]
|   |--- x1 >  -0.41
|   |   |--- value: [0.14]
|--- x1 >  -0.30
|   |--- x1 <= 0.27
|   |   |--- value: [0.03]
|   |--- x1 >  0.27
|   |   |--- value: [0.15]
```



شجرة الـ regression الحقيقية: كل ورقة فيها `value` = متوسط الـ target للعينات اللي وصلتها (هو ده التنبؤ).



تنبؤ الشجرة = **دالة سلّمية** (قيمة ثابتة جوه كل ورقة). depth=2: 4 أوراق (R²=0.80)، depth=3: 8 أوراق (R²=0.89)، depth=6: 54 ورقة (R²=0.96) بس بتتبع الـ noise (overfitting).

إضافة

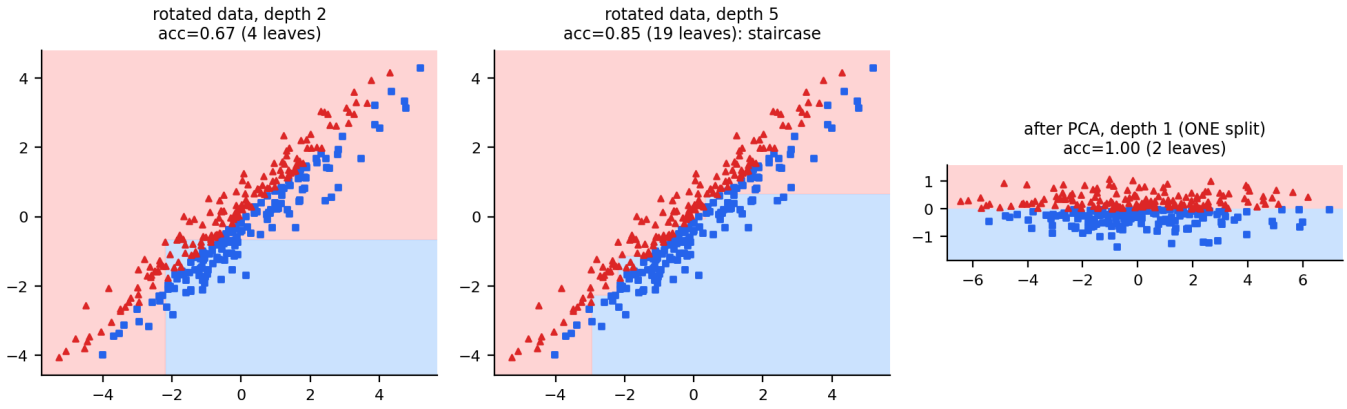
نفس مبدأ الـ regularization في القسم 9 بينطبق هنا: `min_samples_leaf` و`max_depth` ... وكمان الـ Regression tree **مش بيعمل extrapolation**: برا مدى الداتا (مثلًا x=2) بيرجّع قيمة آخر ورقة (ثابتة)، عكس الـ Linear Regression اللي بيكمل الخط.

12 الحساسية لاتجاه المحاور (Sensitivity to Axis Orientation)

من كشكولك

من نقاط الضعف المهمة في Decision Tree إنها تميل إلى بناء **حدود قرار متعامدة على المحاور**، أي تقسيمات رأسية وأفقية عند العمل بميزتين. هذا قد يجعل فصل البيانات أكثر صعوبة.

→ تخيل إن عندك بيانات يسهل فصلها **بخط مائل واحد**. قد تحتاج الشجرة إلى **عدة تقسيمات رأسية وأفقية** لكي تقترب من الخط المائل، بدلًا من استخدام حد واحد بسيط.



تجربة حقيقية: داتا (synthetic) تتفصل بخط مائل واحد. شمال: شجرة depth=2 دقتها 0.67 بس. نص: depth=5 (19 ورقة) بتوصل لـ 0.85 عن طريق "سلالم". يمين: بعد PCA (تدوير المحاور) **سؤال واحد** (2 أوراق) بيفصلهم 1.00.

من كشكولك

هل يساعد الـ PCA؟ يمكن أن يساعد الـ PCA على تغيير تمثيل البيانات واتجاه محاورها. عندما نطبق PCA على الميزات، نحصل على **مكونات جديدة** مثل `z1` و`z2`. هذه المكونات تمثل **اتجاهات جديدة** في فضاء البيانات.

← في مثال الكتاب، بعد تجهيز البيانات واستخدام PCA، استطاعت الشجرة أن تفصل الفئات باستخدام **feature واحدة** من المكونات الجديدة وهي `z1`. لكن يجب الانتباه إلى إن PCA يساعد **غالبًا، وليس دائمًا**، فليس مضمونًا إن تحسين اتجاه البيانات سيؤدي إلى شجرة أفضل في كل Dataset.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pca_pipeline = make_pipeline(StandardScaler(), PCA(n_components=2))
X_iris_pca = pca_pipeline.fit_transform(X_iris)

tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_pca, y_iris)
print(tree_clf_pca.score(X_iris_pca, y_iris), pca_pipeline[-1].explained_variance_ratio_)
print("root split on component", tree_clf_pca.tree_.feature[0] + 1, "<=",
round(tree_clf_pca.tree_.threshold[0], 2))
```

Output (real run)

```
0.9800 [0.9814 0.0186]
root split on component 1 <= -0.91
```

من كشكولك

← استخدمنا `StandardScaler` قبل `PCA` لأن PCA يتأثر بتجهيز الميزات قبل حساب المكونات، ثم دربنا شجرة على البيانات بعد تحويلها. هذا **مثال يوضح تأثير التمثيل الجديد للبيانات** ولا يعني إن كل Decision Tree تحتاج PCA أو Scaling كي تعمل.

إضافة

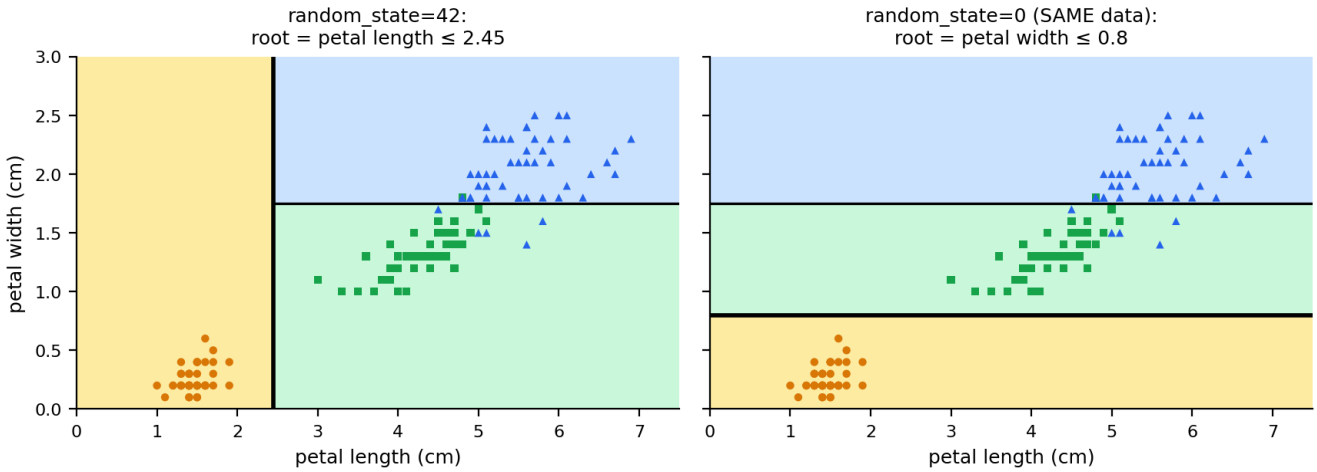
وهنا ينفع نربط بين الملحوظتين: الشجرة **نفسها** مش محتاجة scaling (القسم 3)، لكن **PCA قبلها** هو اللي محتاج. على Iris الحقيقية: دقة الشجرة بعد PCA = **0.980** مقابل **0.960** على الـ features الأصلية. يعني **مفيش تحسين** هنا (الداتا أصلًا بتتفصل بحدود متعامدة تقريبًا)، وده تأكيد لتحذيرك "غالبًا مش دايمًا". المكون الأول لوحده شايل **98%** من التباين.

13 شجرة القرار لها تباين عالٍ (High Variance)

من كشكولك

المقصود هنا إن شجرة القرار **قد تتغير بشكل ملحوظ** عند حدوث **تغييرات صغيرة في بيانات التدريب** أو عند اختلاف **الاختيارات التي تؤثر في بناء الشجرة**. فقد يتغير ترتيب الأسئلة والفروع وبالتالي تتغير حدود القرار.

← هذا يفسر لماذا قد تحصل على **أشجار مختلفة رغم إن البيانات متشابهة جدًا**. ويمكن استخدام `random_state` لجعل التدريب **قابلًا لإعادة الإنتاج** في الظروف نفسها، لكنه **لا يجعل الخوارزمية مستقرة** من شكل الشجرة عندما تتغير بيانات التدريب فعليًا.



تجربة حقيقية على نفس البيانات بالظبط، بس بتغيير `random_state` (شمال 42، يمين 0): الـ root اتغيّر من `petal length` لـ `petal width` وحدود القرار اتغيرت تمامًا. جربت 30 قيمة لـ `random_state` : **16** اختاروا petal width و**14** اختاروا petal length.

إضافة

(ده التعادل اللي اتكلمنا عنه في القسم 6: الاتنين بيدّوا نفس ل.) وفيه تجربة تانية صريحة: شلت زهرة واحدة (versicolor عند 4.8 × 1.8، اللي الكتاب بيستخدمها كمثال): في شجرة `max_depth=2` **مفيش تغيير** ظهر (الشجرة نفسها: True)، لكن في شجرة **غير مقيدة** على الـ petal features عدد الأوراق اتغيّر من **8** لـ **7** والقواعد اختلفت (نفس القواعد؟ False). يعني التباين ببان أكتر كل ما الشجرة تكبر، وده بالظبط بيرجعنا لـ regularization (القسم 9).

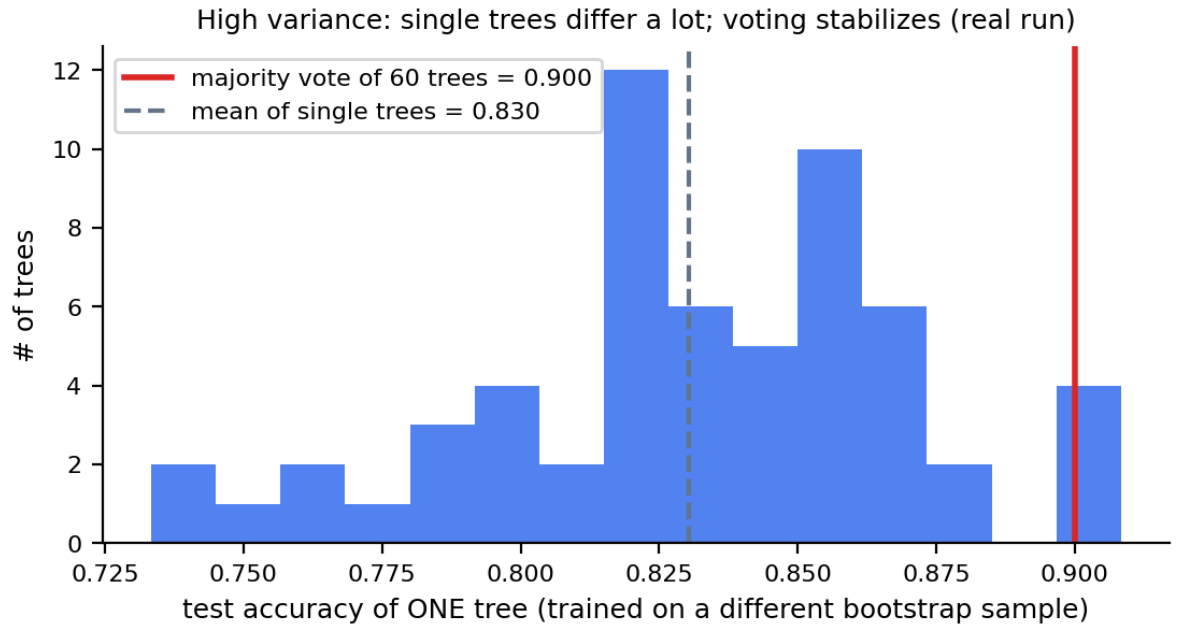
من كشكولك

كيف نقلل أثر هذه المشكلة؟ إحدى الطرق المهمة هي **تجميع تنبؤات عدة أشجار** بدل الاعتماد على شجرة واحدة في التصنيف مثلًا. هذه هي **الفكرة الأساسية وراء الـ Random Forest** الذي سندرسه في الشابتر التالي: تجميع الأشجار قد **يقلل التباين** الناتج عن حساسية الشجرة الواحدة.

```
rng = np.random.RandomState(5); preds, accs = [], []
for _ in range(60):
    idx = rng.randint(0, len(X_tr), len(X_tr))                    # a bootstrap sample (random sample
WITH replacement)
    tree = DecisionTreeClassifier(random_state=0).fit(X_tr[idx], y_tr[idx])
    accs.append(tree.score(X_te, y_te)); preds.append(tree.predict(X_te))
vote = (np.mean(preds, axis=0) > 0.5).astype(int)                 # majority vote of the 60 trees
print(np.mean(accs), np.std(accs), min(accs), max(accs)); print(accuracy_score(y_te, vote))
```

Output (real run)

```
0.830 0.039 0.733 0.908
0.900
```



تجربة حقيقية (60 moons، 60 شجرة، كل واحدة على عينة bootstrap مختلفة): دقة الشجرة الواحدة بتتراوح من 0.73 لـ 0.91 (متوسط 0.83) = high variance. لكن **تصويت الـ 60 شجرة** (الخط الأحمر) = 0.90، أحسن من أي شجرة تقريبًا. دي فكرة Random Forest اللي جاية.

14 أسئلة مراجعة سريعة

جاوب الأول في دماغك، وبعدين شوف الإجابات في الصفحة اللي بعدها.

1. إيه هو الـ Decision Tree؟ وإمتى بيُستخدم في Classification وإمتى في Regression؟
2. إيه معنى max_depth و fit في كود Iris؟
3. إيه الفرق بين Root و Node و Leaf؟
4. إيه معنى samples و value و gini في كل عقدة؟
5. اكتب معادلة Gini. وإيه قيمتها لعقدة نقية؟ واحسبها لـ [0, 49, 5].
6. إيه هي الـ Decision Boundaries؟ وإيه اللي بيتغيّر لما نزوّد max_depth؟
7. ليه الشجرة مش محتاجة feature scaling؟
8. إيه الفرق بين White Box و Black Box؟ وهل قراءة الشجرة بتضمن إنها عادلة؟
9. إزاي الشجرة بتحسب predict_proba؟ وإيه اللي بيحصل لنقطتين في نفس الورقة؟
10. إيه هي خوارزمية CART؟ واكتب دالة تكلفتها للـ classification.
11. ليه CART اسمها Binary Tree؟ وإيه الفرق مع ID3؟
12. يعني إيه Greedy؟ وليه CART مبتجربش كل الأشجار الممكنة؟
13. إيه التعقيد الحسابي للتنبؤ وللتدريب؟ وإمتى بيبقى التنبؤ أبطأ؟
14. إيه هي الـ Entropy؟ والفرق بينها وبين Gini؟
15. اذكر 6 hyperparameters لتقليل overfitting في الشجرة.
16. ليه شجرة من غير قيود بتعمل overfitting؟ وهل القيود بترفع الدقة دايمًا؟
17. إيه هو الـ Pruning؟ وإيه دور اختبار χ^2 والـ p-value؟
18. إيه اللي بتطلّعه كل ورقة في DecisionTreeRegressor؟
19. اكتب دالة تكلفة CART للـ regression.
20. ليه Decision Tree حساسة لاتجاه المحاور؟ وإزاي ممكن PCA يساعد؟ وهل بيساعد دايمًا؟
21. إيه معنى إن Decision Tree لها high variance؟ وإيه علاقة random_state؟
22. إزاي نقلل الـ variance؟ وإيه علاقة ده بـ Random Forest؟

15 إجابات أسئلة المراجعة

1. موديل بيسأل أسئلة متتابعة عن الـ features وكل إجابة بتودّي لفرع لحد قرار نهائي. Classification: بيتنبأ بفئة. Regression: بيتنبأ برقم (متوسط الـ target في الورقة).
2. max_depth = أقصى عمق للشجرة (لتبسيطها). fit(X, y) = التدريب: الشجرة بتدوّر على أحسن الأسئلة اللي بتفصل الأنواع.
3. Root: أول عقدة. Node: عقدة فيها اختبار/قرار. Leaf: نهاية الفرع ومنها التنبؤ.
4. samples = عدد عينات التدريب اللي وصلت العقدة. value = عدد العينات من كل class. gini = مدى اختلاط الفئات (0 = نقية).
5. $G = 1 - \Sigma\, p^2$. لعقدة نقية = 0. لـ [0,49,5]: $1 - (49/54)^2 - (5/54)^2 = 0.168$.
6. Decision boundaries = المناطق اللي الشجرة بتقسم بيها الفراغ (مستطيلات). زيادة max_depth = أسئلة أكتر = مناطق أصغر وحدود أعقد.
7. لأن كل سؤال بيقارن feature واحدة بـ threshold (مثل petal length ≤ 2.45) ومبيعتمدش على المسافة أو المقياس.
8. White Box: نقدر نقرا الأسئلة والفروع اللي أدّت للقرار. Black Box: التفسير صعب. لكن سهولة القراءة لا تضمن إن القرارات صحيحة أو عادلة.
9. النسبة داخل الورقة: samples الفئة ÷ samples الورقة. أي نقطتين في نفس الورقة ليهم نفس الاحتمالات.
10. CART = Classification and Regression Tree: في كل عقدة تختار feature و threshold يقللوا $J = (m_{left}/m)\cdot G_{left} + (m_{right}/m)\cdot G_{right}$.
11. لأن كل تقسيم بيعطي فرعين بس. ID3 (في صورته التقليدية) ممكن يطلّع أكتر من فرعين حسب قيم الـ feature.
12. Greedy = بتختار أحسن تقسيم للعقدة الحالية بس. إيجاد الشجرة المثلى (O(exp(m))) NP-complete فمستحيل عمليًا.
13. التنبؤ: $O(\log_2 m) \approx$ للشجرة المتوازنة (يعتمد على العمق الفعلي، وأبطأ لو الشجرة غير متوازنة). التدريب: $O(n \times m \approx n \log m)$.
14. $\text{Entropy} = -\Sigma\, p \log_2 p$، بتقيس عدم اليقين (0 للعقدة النقية). Gini أبسط حسابيًا؛ الاتنين غالبًا بيدّوا أشجار متشابهة لكن مش دايمًا نفس الشكل.
15. max_depth, min_samples_split, min_samples_leaf, min_weight_fraction_leaf, max_leaf_nodes, max_features.
16. لأنها بتتعلم تفاصيل وضوضاء بيانات التدريب. القيود مش دايمًا بترفع الدقة، بس بتمنع بعض التقسيمات فتحسّن التعميم (وزيادتها أكتر من اللازم = underfitting).
17. Pruning = تسيب الشجرة تكبر وبعدين تحذف الأفرع اللي مش بتحسّن النقاء بدلالة إحصائية. χ^2 يقيّم هل التحسن صدفة، وp-value أكبر من حد (5%) يعني حذف العقدة وأبنائها. (sklearn بتستخدم ccp_alpha بدلًا منه.)
18. كل ورقة بتطلّع متوسط قيم الـ target لعينات التدريب اللي وصلتها.
19. $J = (m_{left}/m)\cdot MSE_{left} + (m_{right}/m)\cdot MSE_{right}$، حيث MSE_node متوسط $(y - \hat{y}_{node})^2$ و $\hat{y}_{node}$ متوسط الـ target في العقدة.
20. لأنها بتعمل حدود متعامدة على المحاور (رأسية/أفقية)، فبتحتاج تقسيمات كتير عشان تقرّب خط مائل. PCA بيدوّر المحاور فممكن سؤال واحد يكفي، لكن مش دايمًا بيحسّن (على Iris ما حسّنش).

21. إن الشجرة بتتغير كتير مع تغييرات صغيرة في الداتا أو في اختيارات البناء (زي كسر التعادل). random_state بيخلّي التدريب قابل للإعادة لكن مش بيخلّي الشجرة مستقرة لما الداتا تتغير.

22. بتجميع تنبؤات أشجار كتير (تصويت) بدل شجرة واحدة، وده أساس Random Forest في الشابتر الجاي.